

Retrofitting a Desk Lamp into an IoT-Controlled Device: An End-to-End AWS IoT Core Implementation with Local Sensing and Cloud-Managed State

Matheus Latorre Cavini

*Departamento de Engenharia de Sistemas Eletrônicos
Escola Politécnica da Universidade de São Paulo (EPUSP)*

São Paulo, Brasil

matheuslcavini@usp.br

Abstract—This work presents the design and implementation of an end-to-end Internet of Things (IoT) system built around the retrofit of a desk lamp whose physical switch is broken and therefore remains permanently powered when plugged in. An ESP32 microcontroller, a relay module, a photoresistor and an ultrasonic distance sensor are combined to turn the lamp into a connected device capable of both autonomous and remotely commanded operation. The device authenticates to AWS IoT Core over mutual TLS using an X.509 certificate, publishes telemetry over MQTT, and exposes its operating mode and lamp state through an AWS IoT Device Shadow, which is used as the single mechanism for cloud-to-device state synchronization. Telemetry is routed through an IoT Rule to an AWS Lambda function that persists each reading in Amazon DynamoDB, and a second Lambda function, exposed through an HTTP API, backs a browser-based control panel that reads and updates the shadow. The system is evaluated under four experimental scenarios: autonomous (sensor-driven) operation, manual (shadow-driven) operation, a stateless identification command, and end-to-end data persistence. Results show shadow convergence within a few seconds of a desired-state change, a measured average Lambda execution time of 35.7 ms for warm invocations (761 ms billed on the initial cold start), and a total estimated AWS cost, computed from measured message and invocation counts against published on-demand pricing, on the order of a few US cents for the entire development and testing period.

Index Terms—Internet of Things, IoT platform, AWS IoT Core, MQTT, Device Shadow, ESP32, DynamoDB, edge computing

I. INTRODUCTION

Connecting a physical device to cloud infrastructure introduces a set of architectural concerns that do not appear when a cloud application communicates only with other cloud services or with a conventional web client. A device may be intermittently connected, must be authenticated and authorized individually, and typically requires a representation of its state that remains accessible to applications even while the device itself is offline. These concerns are addressed by a dedicated architectural layer commonly referred to as an IoT platform, which sits between the device population and the cloud services responsible for storing, processing and consuming the resulting data [1].

This work applies these concepts to a small but concrete physical problem. A desk lamp has a broken switch and remains powered on continuously whenever it is plugged into an outlet, which removes the ability to turn it off without physically unplugging it. This work retrofits the lamp with an ESP32 microcontroller, a relay module, a photoresistor and an ultrasonic distance sensor, restoring control over the lamp state while adding two new capabilities that did not exist in the original device: autonomous operation based on ambient light and proximity, and remote operation through a cloud platform. The resulting system is used as a vehicle to implement, and subsequently evaluate, the core building blocks of a managed IoT platform: device identity and mutual authentication, MQTT-based telemetry, a persistent shadow-based state model, rule-driven integration with serverless processing, and data persistence.

The device supports two operating modes, selected and observed through an AWS IoT Device Shadow. In *auto* mode, the device decides locally whether the lamp should be on, based on ambient light and proximity readings, and reports the resulting state to the cloud. In *manual* mode, local sensing is ignored for actuation purposes and the lamp state is instead driven by the desired state recorded in the shadow, which an operator changes through the AWS IoT Core console. A third, stateless *identify* command, delivered over a plain MQTT topic rather than through the shadow, is used to contrast a fire-and-forget command with the persistent, convergence-based semantics of shadow state.

The remainder of this article is organized as follows. Section II reviews the background concepts required to justify the architectural choices made in this work: the distinction between an MQTT broker and an IoT platform, MQTT publish/subscribe fundamentals, device identity and X.509-based mutual authentication, and the desired/reported state model implemented by AWS IoT Device Shadow. Section III describes the system architecture, covering both the physical device and the cloud-side pipeline. Section IV details the implementation, including device provisioning, firmware behavior and

the serverless persistence pipeline. Section V presents the experimental evaluation performed on the deployed system. Section VI reports the measured AWS cost of the experiments. Section VII discusses the architectural implications of the design choices made, including vendor lock-in surfaces and the boundary between cloud and edge decision-making. Section VIII concludes the article.

II. BACKGROUND AND RELATED CONCEPTS

A. IoT platforms and MQTT brokers

An MQTT broker implements the server side of the MQTT protocol: it accepts client connections, processes publications and forwards messages to matching subscribers [2]. This is a necessary capability for device connectivity, but it does not by itself provide device registration, certificate-based provisioning, fleet management, persistent device state, or integration with downstream cloud processing. An IoT platform is the broader set of integrated services that, in addition to message brokering, typically provides device identity, authentication and authorization, a device registry, state representation, rule-based routing and integration with external services [3]. AWS IoT Core is used in this work as a concrete instance of a managed IoT platform, providing these capabilities without requiring the operation of the underlying broker, registry or state service infrastructure.

B. MQTT publish/subscribe fundamentals

MQTT is a client-server publish/subscribe messaging protocol standardized by OASIS [4]. A publisher sends a message associated with a hierarchical topic name; a subscriber registers interest in one or more topics, optionally using wildcards, through a topic filter; the broker is responsible for matching publications to subscriptions and delivering messages accordingly. MQTT defines three Quality of Service (QoS) levels for message delivery between two protocol participants, ranging from at-most-once (QoS 0) to a protocol-level exactly-once delivery (QoS 2) [4]. It is worth noting that QoS operates strictly at the level of a single MQTT transfer: it does not, by itself, guarantee that a downstream chain of services processes the resulting event exactly once, since retries and duplicated invocations can still occur past the broker. This distinction motivates treating message identity (through fields such as a monotonic sequence number) as an application-level concern rather than one solved implicitly by the transport protocol.

C. Device identity and mutual TLS authentication

Authentication answers the question of which identity is attempting to connect; authorization determines which operations that identity is permitted to perform [1]. In this work, device identity is established through an X.509 certificate, a structure that binds identity information to a public key and is signed by a trusted certificate authority. Under mutual TLS (mTLS), both the device and the AWS IoT Core endpoint present credentials during the TLS handshake, so that the device validates the identity of the platform and the platform

validates the identity of the device [5]. Authorization is enforced separately, through an IoT policy attached to the device certificate, which specifies exactly which MQTT operations (connect, publish, subscribe, receive) are allowed on which topics and shadow resources, following the principle of least privilege.

D. Device Shadow and the desired/reported state model

A device shadow is a persistent representation of a device's state that decouples cloud applications from the instantaneous availability of the device itself [6]. The shadow document maintains a *desired* state, written by an application or operator, and a *reported* state, written by the device. When the two differ, the platform notifies the device of the difference (a delta), and the device is expected to apply the corresponding change and subsequently update its reported state, at which point the two converge. This convergence is not instantaneous nor guaranteed within a bounded time: a device that is offline, powered down or otherwise disconnected will not receive the delta until it reconnects, so the desired and reported states may remain divergent for an arbitrary interval. This property is exploited directly in this work, where the shadow is used as the single mechanism to represent both the lamp's operating mode and its on/off state, rather than introducing a separate ad hoc state channel.

E. Rules engine and serverless integration

A rules engine evaluates incoming messages against a set of conditions and routes or transforms matching messages toward downstream destinations such as functions, queues, storage or analytics services [7]. In this work, an AWS IoT Rule selects telemetry messages by topic and forwards them, unmodified, to an AWS Lambda function, which persists each reading in Amazon DynamoDB. This pipeline reuses the Function-as-a-Service and event-driven integration pattern already established in the discipline for non-IoT workloads, with the IoT platform acting as the component that authenticates the originating device and exposes its data as a conventional cloud event.

III. SYSTEM ARCHITECTURE

A. Physical device

The physical device is a desk lamp whose original switch is broken and which therefore remains energized continuously once plugged into an outlet. An ESP32 microcontroller [9] drives the retrofit. An opto-isolated relay module is inserted between the outlet and the lamp to allow the ESP32 to switch the AC load under software control. Fig. 1 shows the resulting diagram. Two sensors provide environmental context for autonomous operation: a photoresistor (LDR), read through an analog input to estimate ambient light level, and an HC-SR04 ultrasonic distance sensor, used to detect the proximity of a person near the lamp.

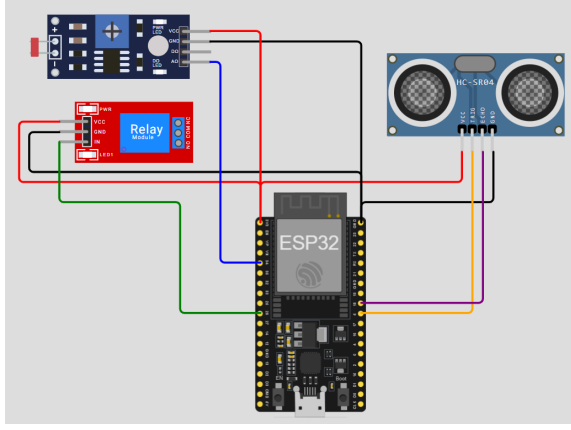


Fig. 1. Schematic diagram of the physical device: ESP32, relay module, photoresistor and ultrasonic sensor connected to the retrofitted lamp.

B. Cloud architecture

Fig. 2 shows the overall pipeline. The ESP32 connects to AWS IoT Core over mTLS, authenticated with an X.509 certificate associated with a single Thing representing the lamp. Telemetry (ambient light reading, proximity flag, current lamp state, a monotonic sequence number and a timestamp) is published periodically to a dedicated MQTT topic. An IoT Rule matches this topic and invokes an AWS Lambda function, which writes each telemetry sample to an Amazon DynamoDB table. Device state (operating mode and lamp state) is represented exclusively through the AWS IoT Device Shadow: the device subscribes to shadow delta notifications, applies the corresponding change locally, and republishes its reported state. A separate, stateless `identify` command is delivered over a plain MQTT topic to illustrate the distinction, introduced in Section II-D, between a transient command and persistent shadow state.

C. State model

The system exposes two pieces of state through the shadow document: `mode`, which takes the values `auto` or `manual`, and `lamp_state`, which takes the values `on` or `off`. In `auto` mode, the firmware computes `lamp_state` locally from the sensor readings, drives the relay accordingly, and reports the resulting value; changes to `desired.lamp_state` are ignored while in this mode. In `manual` mode, sensor readings continue to be published as telemetry but no longer influence the relay; the relay instead follows `desired.lamp_state`, and the device reports the applied value once the change has taken effect. Switching between modes is itself performed through `desired.mode`, using the same `delta/report` mechanism. This design keeps a single source of truth for device-observable state, at the cost of committing every meaningful state transition to the shadow's `desired/reported` convergence model rather than to a lower-latency command-response exchange.

D. Web control panel

In addition to the AWS IoT Core console, a browser-based control panel (Fig. 3) was built to issue commands as an ordinary application would.

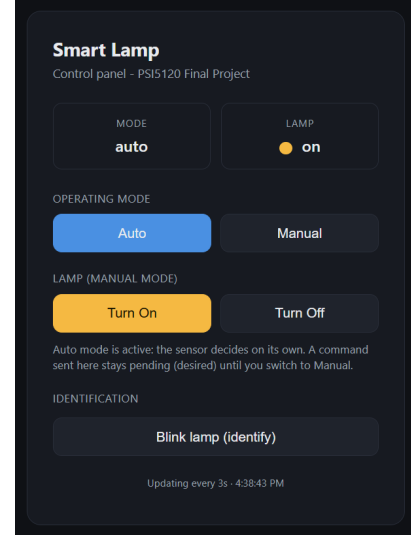


Fig. 3. Web control panel interface

Rather than having the browser speak MQTT directly to AWS IoT Core, a second AWS Lambda function is exposed through an HTTP API and acts as a thin backend: it reads the current shadow document, writes new `desired` values, and publishes the `identify` command, using the AWS IoT data-plane API (boto3's `iot-data` client) rather than an MQTT client. This choice keeps the browser-side code to plain HTML and JavaScript, with no signing or credential material exposed to the client, and reuses the same serverless integration pattern already established for the persistence pipeline in Section IV-C. It also introduces a second, independent instance of the two-authorization-relationship pattern discussed in that section: the HTTP API's permission to invoke this second Lambda function is distinct from that function's own permission, granted through its execution role, to read and write the shadow and to publish to the command topic.

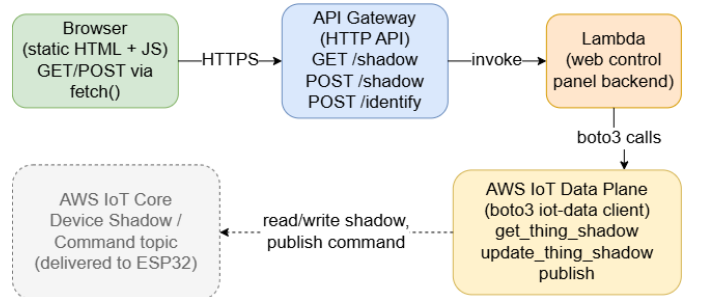


Fig. 4. Cloud-to-device path used by the web control panel: browser, HTTP API, backend Lambda and the AWS IoT data plane.

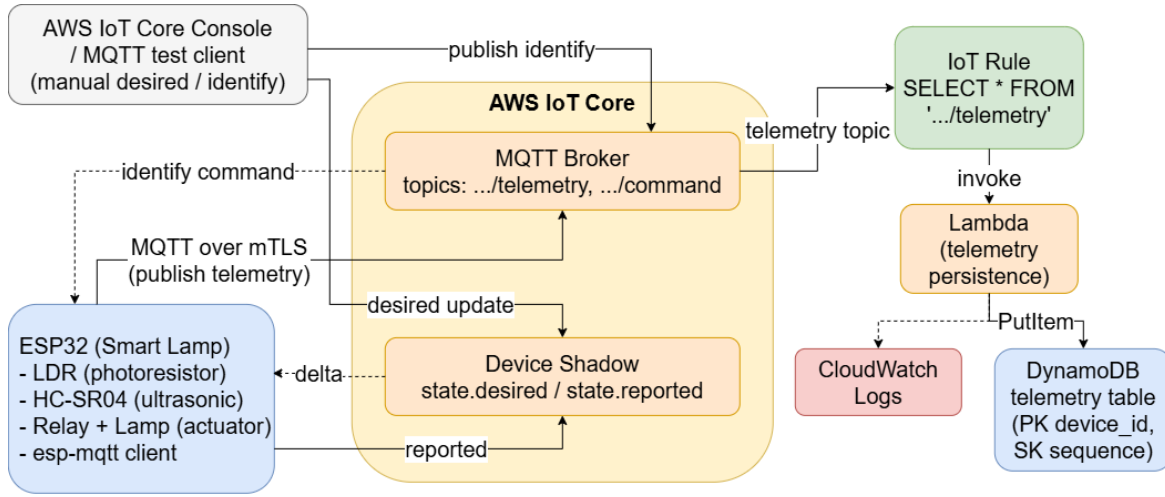


Fig. 2. End-to-end architecture: device, AWS IoT Core (broker and Device Shadow), IoT Rule, Lambda and DynamoDB.

IV. IMPLEMENTATION

A. Device provisioning

A single AWS IoT Thing is created to represent the lamp, together with an X.509 certificate generated by AWS IoT Core and an IoT policy scoped to the exact set of operations the device needs: connecting to the endpoint, publishing telemetry and the `identify` acknowledgment, subscribing to and receiving shadow delta notifications, and updating its reported shadow state. No wildcard resource is used in the policy, so that any operation outside this set is denied by default rather than allowed implicitly.

B. Firmware

The firmware runs a periodic loop with two independent timers: sensor readings and the local *auto*-mode decision are re-evaluated every 500 ms, while a full telemetry message is published every 3 s. The light threshold and the 10 cm proximity threshold used to decide `lamp_state` in *auto* mode were calibrated experimentally against the actual LDR and HC-SR04 readings before any cloud connectivity was introduced, so that the sensing and actuation logic could be validated in isolation. Incoming MQTT messages (the shadow delta and the `identify` command) are handled asynchronously through a callback, independent of the telemetry timer.

The initial firmware paired a TLS-capable socket library with a lightweight MQTT client commonly used in the Arduino/ESP32 ecosystem. This combination connected and authenticated correctly, but exhibited frequent, unexplained mid-session disconnections once the system was under continuous operation; Section VII-C reports the diagnostic process and the resolution, which required replacing the underlying MQTT/TLS stack.

C. Serverless persistence pipeline

The Lambda function receives the telemetry payload forwarded by the IoT Rule and writes it to a DynamoDB table keyed by device identifier (partition key) and sequence number

(sort key), reusing the persistence pattern already established for non-IoT event-driven pipelines in the discipline. This separates two independent authorization relationships that are easy to conflate: the permission for AWS IoT to invoke the Lambda function, granted through the function’s resource-based policy, and the permission for the Lambda function to write to DynamoDB, granted through its execution role.

D. Web control panel backend

The backend behind the control panel described in Section III-D is a second, independent Lambda function, invoked synchronously by an HTTP API rather than asynchronously by an IoT Rule. It exposes three routes: `GET /shadow`, which returns the current shadow document; `POST /shadow`, which accepts a partial `desired` object and writes it to the shadow; and `POST /identify`, which publishes the stateless command directly to the command topic. No end-user authentication is implemented in front of this API, a deliberate scope simplification discussed in Section VII-D.

E. Security considerations

Two independent categories of risk are addressed in this work. At the network and identity layer, mutual TLS with a per-device X.509 certificate and a least-privilege IoT policy limit both who can connect and what an authenticated device is allowed to do once connected – addressing, at device scale, the identification, access-restriction and update-related capabilities described in the NIST device cybersecurity baseline [8]. At the physical layer, all mains-voltage wiring is confined to an enclosed, opto-isolated relay module, and the lamp is not handled while connected to line power during testing, since the retrofit modifies a live AC circuit rather than a low-voltage prototype.

V. EXPERIMENTAL EVALUATION

Evidence for the scenarios below was collected from three independent sources that had to agree with one another: the

device’s serial console, the AWS IoT Core console (Device Shadow view and MQTT test client), and CloudWatch Logs for the Lambda function. Table I summarizes the four scenarios that were executed.

TABLE I
SUMMARY OF EXECUTED EXPERIMENTAL SCENARIOS

Scenario	Key evidence	Result
Autonomous (auto)	Serial log: AUTO_DECISION on darkness and on proximity, independently	Both trigger conditions confirmed
Manual Shadow + identify command	desired → delta → reported sequence; Console accept event COMMAND_RECEIVED / IDENTIFY_DONE, no shadow publish	Converged within one telemetry interval (≤ 3 s) Stateless behavior confirmed
Persistence	CloudWatch + DynamoDB item count	33/33 messages persisted

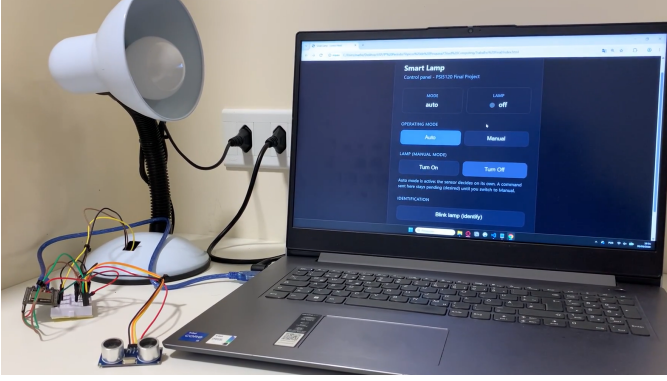


Fig. 5. Full experimental setup, including retrofitted smart lamp and command interface

A. Autonomous operation (auto mode)

With mode set to *auto*, the device was tested against its two independent trigger conditions separately. Covering the LDR produced a transition from `light_raw` 864 (`escuro: true`) with no proximity detected, which the firmware correctly resolved to `lamp_state: on`. In a separate run, approaching a hand to the ultrasonic sensor produced `distance_cm` 4.51 (`presence_proxima: true`) with ambient light unchanged (`escuro: false`), which independently produced the same on transition, confirming that the two conditions are combined with a logical OR rather than one masking the other. Withdrawing the trigger in both cases produced the corresponding off decision and a new reported shadow update. In a later run, while the device was already in *manual* mode, switching `desired.mode` back to *auto* and then attempting to also set `desired.lamp_state` produced the log line `SHADOW_DELTA_IGNORED: lamp_state recebido em modo auto; sensor decide localmente`, confirming that a `desired.lamp_state` received while

in *auto* mode is deliberately ignored rather than silently overwritten, exactly as specified in Section III-C.

B. Manual operation and shadow convergence (manual mode)

Setting `desired.mode` to *manual* through the console produced `SHADOW_DELTA: mode → manual` on the device within the same telemetry interval, followed by a reported update reflecting the new mode. A subsequent `desired.lamp_state: on` was applied similarly, producing `SHADOW_DELTA: lamp_state → on`, an immediate relay actuation, and a reported update. The console’s own shadow/update/accepted event for this change was timestamped 18:39:21 UTC, bracketed by telemetry messages at 18:39:20 (still `lamp_state: off`) and 18:39:23 (already `lamp_state: on`), which bounds the observed convergence latency to at most one telemetry interval (3 s) and is consistent with the shadow delta being delivered over the already-open MQTT connection rather than requiring a new one. Throughout this scenario, the sensor readings continued to be published as ordinary telemetry, confirming that sensing was not disabled, only decoupled from actuation. A secondary observation from this data is that `light_raw` readings increase measurably whenever the lamp itself is on (e.g., readings around 2600–2800 while on, against a baseline near 2550–2560 while off under otherwise similar ambient conditions): the LDR necessarily senses the lamp’s own light output in addition to the room’s ambient light. This coupling has no effect on the *manual* scenario, since sensor readings do not drive actuation in this mode, but it is a relevant consideration for the *auto* mode and is revisited in Section VII-D.

C. Stateless command (identify)

Publishing `{"command": "identify"}` to the command topic produced `COMMAND_RECEIVED: identify` followed by `IDENTIFY_DONE` on the device, with the lamp blinking as implemented. No shadow publish accompanied this command in any of the runs, and the shadow document’s version counter (used by AWS IoT Core to track shadow updates) was unaffected. This confirms the intended contrast with Section IV-C: a command delivered as a plain MQTT message has no persisted state, no delivery confirmation beyond the transport layer, and no effect on the desired/reported convergence model.

D. End-to-end persistence

Over one representative 96-second test window, the device published 33 telemetry messages (sequence numbers 0 through 32), spanning a transition from *auto* mode (sequences 0–24) to *manual* mode (sequences 25–32). All 33 messages produced a matching pair of CloudWatch log entries, `iot_rule_received` followed by `dynamodb_put_ok`, and all 33 corresponding items were confirmed present in the DynamoDB table via Explore table items, with sequence values matching exactly and mode correctly reflecting the transition. The Lambda function’s own execution logs additionally provide latency and resource data: the initial

invocation exhibited a cold start with an Init Duration of 761.18 ms and an execution Duration of 316.64 ms (1078 ms billed in total), while the remaining 32 warm invocations averaged 35.72 ms in execution duration, with a maximum reported memory usage of 88 MB out of the 128 MB allocated. No invocation reported an error.

VI. COST ANALYSIS

Due to the latency between resource usage and its consolidation in AWS Cost Explorer and the AWS Billing dashboard, the figures reported in this section were computed from measured usage (message counts, invocation counts and billed Lambda duration, all taken directly from the evidence described in Section V) multiplied by the published on-demand pricing for the US East (N. Virginia) region as retrieved on September 26, 2026 [10]–[12], rather than read from a billing dashboard. Table II shows this calculation for the 96-second, 33-message window used in Section V-D, which is representative of the request pattern used throughout development and testing.

TABLE II
ESTIMATED AWS COST FOR A REPRESENTATIVE 33-MESSAGE WINDOW

Component	Cost (US\$)
IoT Core – Messaging (33 msgs × \$1.00/M)	0.000033
IoT Core – Rules triggered + actions (33 × 2 × \$0.15/M)	0.000099
Lambda – Requests (33 × \$0.20/M)	0.000066
Lambda – Compute (0.279 GB-s × \$0.0000166667)	0.000047
DynamoDB – Writes (33 × \$1.25/M WRU)	0.000043
Total (this window)	~0.0001

Two components are omitted from Table II but are similarly negligible at the volumes observed in this project: IoT Core connectivity, billed at \$0.08 per million connection-minutes, and Device Shadow operations, billed at \$1.25 per million operations [10] – both apply to quantities (minutes of connection, shadow reads/writes) that remained in the tens to low hundreds across the entire development period. In practice, both AWS Lambda’s permanent free tier (1 million requests and 400,000 GB-seconds of compute per month) and the low absolute volumes involved mean that most of these charges would not appear on an actual bill at all; the figures above represent an on-demand-equivalent upper bound computed unit by unit, which is more conservative than the true cost. Extrapolating this per-message rate to the entire development and testing period – together amounting to a few thousand messages and invocations at most, never sustained continuous traffic at scale – places the total realistic cost of this project, from initial provisioning through the last experiment, at no more than a few US cents. This is consistent with the discipline’s own observation that a serverless, pay-per-use IoT pipeline of this size sits well within experimentation-scale spending, provided idle resources (in this case, none of the

deployed services accrue cost while the device is disconnected) do not accumulate charges of their own; all resources created for this project were deleted after the last experiment, following the same teardown discipline established in the course’s own practical material.

VII. DISCUSSION

A. Vendor lock-in surfaces

The transport layer of this system, plain MQTT over TLS with a standard topic hierarchy, could in principle be ported to a different broker with comparatively little change to the firmware: topic names and payload structure are entirely application-defined and carry no AWS-specific syntax. The same is not true of the remaining components. The Device Shadow’s desired/reported/delta model, its reserved topic namespace and its JSON envelope (including the `version` and `metadata` fields) are specific to AWS IoT Core; a shadow-equivalent construct such as Azure IoT Hub’s device twin exists conceptually but is not wire-compatible. The IoT Rule’s SQL-like selection syntax and its binding to a Lambda action are similarly AWS-specific, as is the resource-based policy that authorizes AWS IoT to invoke that Lambda function, and the IAM execution role that in turn authorizes the Lambda function to call DynamoDB or the AWS IoT data-plane API. In the terms used earlier in the discipline, MQTT and the payload schema sit on the high-portability end of this project’s surface, while the shadow API, the rules engine and the two chained authorization relationships sit on the low-portability end: migrating this system to a different provider would require redesigning state representation and cloud-to-device delivery from the ground up, even though the device’s own MQTT client code would need only its endpoint and certificate updated.

B. Edge versus cloud decision-making

The *auto* mode implemented in this work is, by construction, a small instance of an edge computing decision: the device acts on local sensor readings without waiting for a round trip to the cloud, which is appropriate given the latency and availability requirements of turning on a lamp – a control loop that depended on a cloud round trip for every reading would be both slower and unavailable exactly when connectivity is lost, which is also when local, autonomous behavior is most valuable. Volume is a secondary factor: publishing a decision only when `lamp_state` changes, rather than evaluating it in the cloud on every telemetry message, keeps the great majority of sensor readings out of any billed operation entirely. The cloud layer, in contrast, is responsible for functions that inherently require a broader or more persistent scope than a single device can provide on its own: remote reconfiguration of the operating mode from anywhere, a historical record of telemetry across restarts of both the device and the evaluator, and a control surface (the web panel) that does not depend on physical proximity to the lamp. This project does not yet include a distinct edge tier between the device and AWS IoT Core – the ESP32

plays both roles simultaneously – so it anticipates, without yet implementing, the gateway/edge-platform separation that the discipline addresses in the following lecture.

C. A case study in transport-layer reliability

The first working version of the firmware paired a widely used lightweight MQTT client with a general-purpose TLS-capable socket library, both common choices in the Arduino/ESP32 ecosystem. This combination authenticated correctly and exchanged messages under light load, but exhibited a recurring pattern under sustained operation: the connection would drop every few seconds with a `CLIENT_ERROR` disconnect reason and immediately reconnect, cycling indefinitely. Because this reason code is documented to cover both a payload exceeding the broker’s size limit and a malformed sequence of MQTT control packets [13], and the payloads involved were well under any published limit, the diagnostic process ruled out competing hypotheses one at a time against independent evidence rather than by guessing at a fix: an over-broad IoT policy (temporarily granting a wildcard `iot:Publish` resource) produced no change, ruling out authorization; the device’s measured Wi-Fi signal strength remained consistently strong (-22 to -33 dBm) across every disconnect, ruling out a weak radio link; and AWS IoT Core’s own connectivity logs, once enabled, showed a `Publish-In` event failing at a point in the message sequence that did not correspond to the order in which the firmware had issued its MQTT operations, indicating that the byte stream itself was being corrupted somewhere below the application layer. Replacing the MQTT client and TLS socket library with the MQTT client bundled natively in the ESP-IDF layer underlying the Arduino core, which also reconnects automatically without requiring separate application-level logic, eliminated the disconnects entirely, with no further recurrence observed for the remainder of the project. This episode illustrates, in a very concrete form, the discipline’s broader point that a connectivity layer is not defined only by which protocol it speaks, but by which specific implementation enforces that protocol, and that ruling out authorization and network-layer causes with direct evidence is what makes a transport-layer diagnosis credible rather than a last resort.

D. Limitations

This work has several limitations, stated here rather than left implicit. First, the system was built and evaluated with a single device; no fleet-scale provisioning, bulk certificate issuance or fleet management operation was exercised, so claims about this project’s approach to identity and state do not extend automatically to a population of devices. Second, the telemetry schema is specific to this lamp and was not designed for reuse across device types. Third, the DynamoDB write pattern used here overwrites an existing item whenever the same `device_id/sequence` pair is written twice, with no deduplication or conditional write; this is acceptable at the scale and duration of this project but would not be appropriate for a production pipeline exposed to retries or

duplicate delivery. Fourth, the *auto*-mode decision does not account for the coupling observed in Section V-B, in which the lamp’s own light output measurably affects the LDR reading that feeds the darkness condition; this did not produce visible oscillation in testing, since presence-triggered activation does not depend on the light reading and the darkness condition was tested independently, but a longer-running deployment might behave differently near the light threshold. Finally, the HTTP API backing the web control panel (Section III-D) has no authentication in front of it; this was a deliberate simplification to keep the control panel’s implementation within the scope and time available for this project, and would need to be addressed, for instance through an API key or a proper identity provider, before any exposure beyond a private demonstration.

VIII. CONCLUSION

This work retrofitted a desk lamp with a broken switch into a fully connected IoT device, and used that concrete artifact to implement and evaluate the core building blocks of a managed IoT platform: certificate-based device identity, MQTT telemetry, a Device Shadow used as the single source of truth for cloud-to-device state, rule-driven serverless persistence, and a second cloud-to-device path through an HTTP-backed web control panel. The four executed experimental scenarios confirmed that the intended behavior of each of these mechanisms holds under real hardware and real network conditions, not only in the abstract: the two independent *auto*-mode trigger conditions each activate the lamp on their own, shadow convergence between a console-issued `desired` change and the device’s `reported` state completes within a single telemetry interval, the stateless `identify` command leaves no trace in the shadow’s convergence history, and all telemetry produced during a representative test window was persisted end to end with a warm Lambda execution time under 40 ms. The generalization of these specific numbers is necessarily limited to this device and this workload; the same code exercised at a fleet scale, or against a different network environment, would produce different latency and reliability figures, though the mechanisms themselves are architectural rather than device-specific. The most substantial unplanned result of this project was not a number but a debugging process: the disconnect behavior investigated in Section VII-C showed, in practice rather than in theory, why the discipline’s material insists on separating authentication, authorization and transport-layer failures as distinct hypotheses to be tested individually against evidence, since the actual cause here was none of the two that are usually suspected first. Together with the discussion of vendor lock-in surfaces and the edge/cloud boundary in Sections VII-A and VII-B, this project’s central empirical finding is that a managed IoT platform meaningfully reduces the effort of building each individual capability, while still leaving the responsibility for diagnosing failures across the resulting stack, from certificate to socket to cloud service, with the team operating it.

REFERENCES

- [1] ISO/IEC, *ISO/IEC 30141:2024 – Internet of Things (IoT) – Reference architecture*, 2024. [Online]. Available: <https://www.iso.org/standard/88800.html>
- [2] Eclipse Mosquitto, “Documentation.” [Online]. Available: <https://mosquitto.org/documentation/>
- [3] Amazon Web Services, “What is AWS IoT?,” AWS IoT Core Developer Guide. [Online]. Available: <https://docs.aws.amazon.com/iot/latest/developerguide/what-is-aws-iot.html>
- [4] OASIS, *MQTT Version 5.0*, OASIS Standard, 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [5] Amazon Web Services, “X.509 client certificates,” AWS IoT Core Developer Guide. [Online]. Available: <https://docs.aws.amazon.com/iot/latest/developerguide/x509-client-certs.html>
- [6] Amazon Web Services, “AWS IoT Device Shadow service,” AWS IoT Core Developer Guide. [Online]. Available: <https://docs.aws.amazon.com/iot/latest/developerguide/iot-device-shadows.html>
- [7] Amazon Web Services, “Rules for AWS IoT,” AWS IoT Core Developer Guide. [Online]. Available: <https://docs.aws.amazon.com/iot/latest/developerguide/iot-rules.html>
- [8] NIST, *IoT Device Cybersecurity Capability Core Baseline*, NISTIR 8259A, 2020. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/ir/2020/NIST.IR.8259a.pdf>
- [9] Espressif Systems, “ESP32 Series Datasheet.” [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- [10] Amazon Web Services, “AWS IoT Core Pricing.” [Online]. Available: <https://aws.amazon.com/iot-core/pricing/> (accessed Sep. 26, 2026).
- [11] Amazon Web Services, “AWS Lambda Pricing.” [Online]. Available: <https://aws.amazon.com/lambda/pricing/> (accessed Sep. 26, 2026).
- [12] Amazon Web Services, “Amazon DynamoDB pricing – On-demand capacity.” [Online]. Available: <https://aws.amazon.com/dynamodb/pricing/on-demand/> (accessed Sep. 26, 2026).
- [13] Amazon Web Services, “Life cycle events,” AWS IoT Core Developer Guide. [Online]. Available: <https://docs.aws.amazon.com/iot/latest/developerguide/life-cycle-events.html>

APPENDIX

Complete source of `src/main.cpp`, the final working version described in Sections III–V, using the native ESP-IDF MQTT client as discussed in Section VII-C.

```
#include <Arduino.h>
#include <WiFi.h>
#include <ArduinoJson.h>
#include <time.h>
#include "esp_event.h"
#include "mqtt_client.h"
#include "esp_arduino_version.h"
#include "secrets.h"

// -----
// Pin assignment (same as previous phases)
// -----
const int PIN_LDR = 34;
const int PIN_TRIG = 5;
const int PIN_ECHO = 18;
const int PIN_RELAY = 26;

// -----
// Decision thresholds (calibrated in Phase 1)
// -----
const int LDR_ESCURO_LIMiar = 1500;
const float DISTANCIA_LIMiar_CM = 10.0;
const unsigned long ECHO_TIMEOUT_US = 15000UL;
const bool RELAY_ACTIVE_LOW = true;

const unsigned long SENSOR_LOOP_INTERVAL_MS = 500;
const unsigned long TELEMETRY_INTERVAL_MS = 3000;

// -----
// Application global state
// -----
String currentMode = "auto";
String currentLampState = "off";
unsigned long sequenceCounter = 0;

int ultimaLeituraLdr = 0;
bool ultimoEscuro = false;
float ultimaDistanciaCm = -1.0;
bool ultimaPresenca = false;

unsigned long ultimaLeituraSensores = 0;
unsigned long ultimoPublishTelemetria = 0;

String shadowUpdateTopic;
String shadowDeltaTopic;

esp_mqtt_client_handle_t mqttClient = nullptr;
volatile bool mqttConectado = false;

// -----
// Actuator
// -----
void setRele(bool ligar) {
    bool nivelElettrico = RELAY_ACTIVE_LOW ? !ligar : ligar;
    digitalWrite(PIN_RELAY, nivelElettrico ? HIGH : LOW);
}

// -----
// Ultrasonic sensor
// -----
float lerDistanciaCm() {
    digitalWrite(PIN_TRIG, LOW);
    delayMicroseconds(2);
    digitalWrite(PIN_TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(PIN_TRIG, LOW);

    unsigned long duracaoUs = pulseIn(PIN_ECHO, HIGH, ECHO_TIMEOUT_US);
    if (duracaoUs == 0) {
        return -1.0;
    }
    return (duracaoUs * 0.0343f) / 2.0f;
}

// -----
// Real timestamp via NTP
// -----
String obterTimestampIso8601() {
    time_t agora;
    time(&agora);
    struct tm timeinfo;
    gmtime_r(&agora, &timeinfo);
    char buffer[25];
    strftime(buffer, sizeof(buffer), "%Y-%m-%dT%H:%M:%SZ", &timeinfo);
    return String(buffer);
}

// -----
// Shadow: publishes the reported state (device ground truth)
// -----
void publishShadowReported() {
    if (!mqttConectado) {
        Serial.println("[SHADOW_REPORTED_SKIPPED] MQTT disconnected");
        return;
    }

    StaticJsonDocument<256> doc;
    JsonObject state = doc.createNestedObject("state");
    JsonObject reported = state.createNestedObject("reported");
    reported["mode"] = currentMode;
    reported["lamp_state"] = currentLampState;
```

```

char buffer[256];
size_t n = serializeJson(doc, buffer, sizeof(buffer));

int msgId = esp_mqtt_client_publish(mqttClient, shadowUpdateTopic.c_str(),
    buffer, n, 0, 0);

Serial.print(msgId >= 0 ? "[SHADOW_REPORTED] " : "[SHADOW_REPORTED_FAILED] ");
Serial.println(buffer);
}

// -----
// Decision logic: reads sensors and, in auto mode,
// decides lamp_state (same as previous phases)
// -----
void avaliarSensoresEAtualizarEstado() {
    ultimaLeituraLdr = analogRead(PIN_LDR);
    ultimoEscuro = (ultimaLeituraLdr < LDR_ESCURO_LIMIAR);

    ultimaDistanciaCm = lerDistanciaCm();
    ultimaPresenca = (ultimaDistanciaCm > 0 && ultimaDistanciaCm <=
        DISTANCIA_LIMIAR_CM);

    if (currentMode == "auto") {
        String novoEstado = (ultimoEscuro || ultimaPresenca) ? "on" : "off";
        if (novoEstado != currentLampState) {
            currentLampState = novoEstado;
            setRele(currentLampState == "on");
            Serial.print("[AUTO_DECISION] lamp_state -> ");
            Serial.println(currentLampState);
            publishShadowReported();
        }
    }
    // In manual mode, sensors keep being read and published as
    // telemetry, but they do not drive the relay: the shadow decides.
}

// -----
// Telemetry
// -----
void publicarTelemetria() {
    if (!mqttConectado) {
        Serial.println("[TELEMETRY_PUBLISH_SKIPPED] MQTT disconnected");
        return;
    }

    StaticJsonDocument<384> doc;
    doc["device_id"] = THING_NAME;
    doc["sequence"] = sequenceCounter++;
    doc["timestamp"] = obterTimestampIso8601();
    doc["light_raw"] = ultimaLeituraLdr;
    doc["escuro"] = ultimoEscuro;
    if (ultimaDistanciaCm > 0) {
        doc["distance_cm"] = ultimaDistanciaCm;
    } else {
        doc["distance_cm"] = nullptr;
    }
    doc["presence_proxima"] = ultimaPresenca;
    doc["mode"] = currentMode;
    doc["lamp_state"] = currentLampState;

    char buffer[384];
    size_t n = serializeJson(doc, buffer, sizeof(buffer));

    int msgId = esp_mqtt_client_publish(mqttClient, TOPIC_TELEMETRY, buffer, n, 0,
        0);

    Serial.print(msgId >= 0 ? "[TELEMETRY_PUBLISH_OK] " :
        "[TELEMETRY_PUBLISH_FAILED] ");
    Serial.println(buffer);
}

// -----
// Shadow: handles the received delta (desired vs reported diff)
// -----
void handleShadowDelta(const uint8_t* payload, size_t length) {
    StaticJsonDocument<512> doc;
    DeserializationError err = deserializeJson(doc, payload, length);
    if (err) {
        Serial.print("[SHADOW_DELTA_PARSE_ERROR] ");
        Serial.println(err.c_str());
        return;
    }

    JsonObject state = doc["state"];
    bool precisaReportar = false;

    if (state.containsKey("mode")) {
        String novoModo = state["mode"].as<String>();
        if (novoModo == "auto" || novoModo == "manual") {
            currentMode = novoModo;
            precisaReportar = true;
            Serial.print("[SHADOW_DELTA] mode -> ");
            Serial.println(currentMode);
        } else {
            Serial.print("[SHADOW_DELTA_IGNORED] invalid mode value: ");
            Serial.println(novoModo);
        }
    }

    if (state.containsKey("lamp_state")) {
        String novoEstado = state["lamp_state"].as<String>();
        if (currentMode == "manual") {
            if (novoEstado == "on" || novoEstado == "off") {
                currentLampState = novoEstado;
                setRele(currentLampState == "on");
                precisaReportar = true;
                Serial.print("[SHADOW_DELTA] lamp_state -> ");
                Serial.println(currentLampState);
            } else {
                Serial.print("[SHADOW_DELTA_IGNORED] invalid lamp_state value: ");
            }
        }
    }
}

```

```

        Serial.println(novoEstado);
    }
} else {
    Serial.println("[SHADOW_DELTA_IGNORED] lamp_state received in auto mode;
        sensor decides locally");
}
}

if (precisaReportar) {
    publishShadowReported();
}
}

// -----
// Identify command: fire-and-forget, no persisted state
// -----
void handleCommand(const uint8_t* payload, size_t length) {
    StaticJsonDocument<256> doc;
    DeserializationError err = deserializeJson(doc, payload, length);
    if (err) {
        Serial.print("[COMMAND_PARSE_ERROR] ");
        Serial.println(err.c_str());
        return;
    }

    String comando = doc["command"].as<String>();
    Serial.print("[COMMAND_RECEIVED] ");
    Serial.println(comando);

    if (comando == "identify") {
        bool estadoAnterior = (currentLampState == "on");
        for (int i = 0; i < 3; i++) {
            setRele(!estadoAnterior);
            delay(200);
            setRele(estadoAnterior);
            delay(200);
        }
        Serial.println("[IDENTIFY_DONE]");
    } else {
        Serial.println("[COMMAND_IGNORED] unknown command");
    }
}

// -----
// esp-mqtt event handler. Runs in the library's own
// internal task context (not in the main loop()) - kept
// simple, reusing the handling functions defined above.
// -----
static void mqttEventHandler(void* handlerArgs, esp_event_base_t base, int32_t
    eventId, void* eventData) {
    auto event = (esp_mqtt_event_handle_t)eventData;

    switch ((esp_mqtt_event_id_t)eventId) {
        case MQTT_EVENT_CONNECTED:
            Serial.println("[MQTT_CONNECTED]");
            mqttConectado = true;
            esp_mqtt_client_subscribe(mqttClient, shadowDeltaTopic.c_str(), 0);
            esp_mqtt_client_subscribe(mqttClient, TOPIC_COMMAND, 0);
            Serial.println("[SUBSCRIBED] shadow/update/delta and command");
            publishShadowReported();
            break;

        case MQTT_EVENT_DISCONNECTED:
            Serial.println("[MQTT_DISCONNECTED] (esp-mqtt internal auto-reconnect)");
            mqttConectado = false;
            break;

        case MQTT_EVENT_DATA: {
            String topicStr(event->topic, event->topic_len);
            if (topicStr == shadowDeltaTopic) {
                handleShadowDelta((const uint8_t*)event->data, event->data_len);
            } else if (topicStr == String(TOPIC_COMMAND)) {
                handleCommand((const uint8_t*)event->data, event->data_len);
            }
            break;
        }

        case MQTT_EVENT_ERROR:
            Serial.println("[MQTT_EVENT_ERROR]");
            if (event->error_handle && event->error_handle->error_type ==
                MQTT_ERROR_TYPE_TCP_TRANSPORT) {
                Serial.print("[MQTT_TLS_ERROR] esp_tls_last_esp_err=0x");
                Serial.println(event->error_handle->esp_tls_last_esp_err, HEX);
            }
            break;

        default:
            break;
    }
}

// -----
// Connectivity
// -----
void conectarWifi() {
    Serial.print("[WIFI] Connecting to ");
    Serial.println(WIFI_SSID);

    WiFi.mode(WIFI_STA);
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println();
    Serial.print("[WIFI_CONNECTED] Local IP: ");
    Serial.println(WiFi.localIP());
    Serial.print("[WIFI_RSSI] ");
    Serial.println(WiFi.RSSI());
}

```

```

void iniciarMqtt() {
    shadowUpdateTopic = String("$aws/things/") + THING_NAME + "/shadow/update";
    shadowDeltaTopic = shadowUpdateTopic + "/delta";

    String brokerUri = String("mqttps://") + AWS_IOT_ENDPOINT + ":8883";

    esp_mqtt_client_config_t mqttCfg = {};

    #if ESP_ARDUINO_VERSION_MAJOR >= 3
    // arduino-esp32 3.x (ESP-IDF 5.x based): nested config struct
    mqttCfg.broker.address.uri = brokerUri.c_str();
    mqttCfg.broker.verificatio.certificate = AWS_ROOT_CA;
    mqttCfg.credentials.client_id = THING_NAME;
    mqttCfg.credentials.authentication.certificate = DEVICE_CERT;
    mqttCfg.credentials.authentication.key = DEVICE_PRIVATE_KEY;
    #else
    // arduino-esp32 2.x (ESP-IDF 4.4.x based, the case for the
    // espressif32@6.9.0 platform pin used in this project): flat config
    mqttCfg.uri = brokerUri.c_str();
    mqttCfg.cert_pem = AWS_ROOT_CA;
    mqttCfg.client_id = THING_NAME;
    mqttCfg.client_cert_pem = DEVICE_CERT;
    mqttCfg.client_key_pem = DEVICE_PRIVATE_KEY;
    #endif

    mqttClient = esp_mqtt_client_init(&mqttCfg);
    esp_mqtt_client_register_event(mqttClient,
        (esp_mqtt_event_id_t)ESP_EVENT_ANY_ID, mqttEventHandler, NULL);
    esp_mqtt_client_start(mqttClient);

    Serial.println("[MQTT] Client started (esp-mqtt internal auto-reconnect)");
}

// -----
// Setup / Loop
// -----
void setup() {
    Serial.begin(115200);
    delay(300);
    Serial.println();
    Serial.println("[BOOT] Phase 3 (v2, esp-mqtt) - telemetry, shadow, command");

    pinMode(PIN_TRIG, OUTPUT);
    pinMode(PIN_ECHO, INPUT);
    pinMode(PIN_RELAY, OUTPUT);
    digitalWrite(PIN_TRIG, LOW);
    setRele(false);

    conectarWifi();

    esp_err_t evLoopErr = esp_event_loop_create_default();
    if (evLoopErr != ESP_OK && evLoopErr != ESP_ERR_INVALID_STATE) {
        Serial.print("[EVENT_LOOP_ERROR] ");
        Serial.println(esp_err_to_name(evLoopErr));
    }

    configTime(0, 0, "pool.ntp.org", "time.nist.gov");
    Serial.print("[NTP] Syncing time");
    time_t agora = time(nullptr);
    int tentativas = 0;
    const time_t EPOCH_MINIMO_PLAUSIVEL = 8 * 3600 * 2;
    while (agora < EPOCH_MINIMO_PLAUSIVEL && tentativas < 20) {
        delay(250);
        Serial.print(".");
        agora = time(nullptr);
        tentativas++;
    }
    Serial.println();
    Serial.println(agora >= EPOCH_MINIMO_PLAUSIVEL ? "[NTP_OK]" : "[NTP_TIMEOUT]
    proceeding anyway");

    iniciarMqtt();

    // Initial sensor reading. The first publishShadowReported() already
    // happens inside the MQTT_EVENT_CONNECTED event (mqttEventHandler),
    // so it is not repeated here.
    avaliarSensoresEAtualizarEstado();
}

void loop() {
    unsigned long agora = millis();

    if (agora - ultimaLeituraSensores >= SENSOR_LOOP_INTERVAL_MS) {
        avaliarSensoresEAtualizarEstado();
        ultimaLeituraSensores = agora;
    }

    if (agora - ultimoPublishTelemetria >= TELEMETRY_INTERVAL_MS) {
        publicarTelemetria();
        ultimoPublishTelemetria = agora;
    }

    delay(20);
}

```

A. Telemetry Persistence Function

Invoked by the IoT Rule described in Section IV-C; writes each telemetry message to DynamoDB.

```

"""
PSI5120 - Final Project - Smart Lamp Retrofit
Lambda invoked by the AWS IoT Rule on every telemetry message.

Flow: MQTT (telemetry) -> IoT Rule -> this Lambda -> DynamoDB (PutItem)

```

Two distinct, independent authorization relationships make this pipeline work:

1. AWS IoT -> Lambda: controlled by the function's resource-based policy (usually created automatically by the Console when the Lambda action is added to the Rule).
2. Lambda -> DynamoDB: controlled by the Lambda execution role (inline policy with dynamodb:PutItem, set up in a later step).

```

import json
import os
from decimal import Decimal

import boto3

dynamodb = boto3.resource("dynamodb")
table = dynamodb.Table(os.environ["TABLE_NAME"])

def lambda_handler(event, context):
    print(json.dumps({
        "phase": "iot_rule_received",
        "request_id": context.aws_request_id,
        "event": event,
    }, ensure_ascii=False, sort_keys=True))

    item = {
        "device_id": str(event["device_id"]),
        "sequence": int(event["sequence"]),
        "timestamp": str(event["timestamp"]),
        "mode": str(event["mode"]),
        "lamp_state": str(event["lamp_state"]),
        "escuro": bool(event["escuro"]),
        "presence_proxima": bool(event["presence_proxima"]),
        "light_raw": int(event["light_raw"]),
    }

    # distance_cm may arrive as JSON null (no valid HC-SR04 echo on
    # that reading). The boto3 resource does not accept None directly
    # as an attribute value, so the field is only included when present.
    distance_cm = event.get("distance_cm")
    if distance_cm is not None:
        item["distance_cm"] = Decimal(str(distance_cm))

    table.put_item(Item=item)

    print(json.dumps({
        "phase": "dynamodb_put_ok",
        "request_id": context.aws_request_id,
        "device_id": item["device_id"],
        "sequence": item["sequence"],
    }, ensure_ascii=False, sort_keys=True))

    return {
        "statusCode": 200,
        "processed": True,
        "device_id": item["device_id"],
        "sequence": item["sequence"],
    }

```

B. Web Control Panel Backend Function

Invoked by the HTTP API described in Section III-D; reads and updates the Device Shadow and publishes the identify command.

```

"""
PSI5120 - Final Project - Smart Lamp Retrofit
Backend for the web control panel (invoked via API Gateway HTTP API).

Expected routes (API Gateway HTTP API, payload format 2.0):
GET /shadow -> returns the current Device Shadow document
POST /shadow -> updates the Shadow "desired" (body: {"desired": {...}})
POST /identify -> publishes the stateless "identify" command

```

Two distinct authorization relationships make this backend work (same conceptual pattern as the Phase 4 Lambda, now in the cloud-to-device direction instead of device-to-cloud):

1. API Gateway -> Lambda: invocation permission (configured automatically by the Console when the integration is created).
2. Lambda -> AWS IoT Data Plane: controlled by the Lambda execution role (inline policy with iot:GetThingShadow, iot:UpdateThingShadow and iot:Publish, set up in a separate step).

```

import json
import os

import boto3

# IOT_DATA_ENDPOINT example: xxxxxxxxxxxx-ats.iot.us-east-1.amazonaws.com
IOT_DATA_ENDPOINT = os.environ["IOT_DATA_ENDPOINT"]
THING_NAME = os.environ["THING_NAME"]
COMMAND_TOPIC = os.environ["COMMAND_TOPIC"]

iot_data = boto3.client("iot-data", endpoint_url=f"https://{IOT_DATA_ENDPOINT}")

CORS_HEADERS = {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Headers": "Content-Type",
    "Access-Control-Allow-Methods": "GET,POST,OPTIONS",
}

def _response(status_code, body_dict):
    return {

```

```

        "statusCode": status_code,
        "headers": {**CORS_HEADERS, "Content-Type": "application/json"},
        "body": json.dumps(body_dict, ensure_ascii=False),
    }

def lambda_handler(event, context):
    route = event.get("routeKey", "")
    method = event.get("requestContext", {}).get("http", {}).get("method", "")

    # Defensive fallback: CORS preflight is normally handled by the
    # API Gateway's own CORS configuration, without invoking the Lambda.
    if method == "OPTIONS":
        return _response(200, {"ok": True})

    if route == "GET /shadow":
        resp = iot_data.get_thing_shadow(thingName=THING_NAME)
        payload = json.loads(resp["payload"].read())
        return _response(200, payload)

    if route == "POST /shadow":
        body = json.loads(event.get("body") or "{}")
        desired = body.get("desired")
        if not isinstance(desired, dict) or not desired:
            return _response(400, {"error": "body must contain a non-empty
            'desired' object"})

        shadow_payload = json.dumps({"state": {"desired":
        desired}}).encode("utf-8")
        iot_data.update_thing_shadow(thingName=THING_NAME, payload=shadow_payload)
        return _response(200, {"ok": True, "desired": desired})

    if route == "POST /identify":
        command_payload = json.dumps({"command": "identify"}).encode("utf-8")
        iot_data.publish(topic=COMMAND_TOPIC, qos=0, payload=command_payload)
        return _response(200, {"ok": True})

    return _response(404, {"error": f"unknown route: {route}"})

```

C. Device IoT Policy (least privilege)

Attached to the device's X.509 certificate; the final, least-privilege version used for the experiments in Section V, after the temporary wildcard resource used during the diagnostic process in Section VII-C was reverted.

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "Connect",
      "Effect": "Allow",
      "Action": "iot:Connect",
      "Resource": "arn:aws:iot:__REGION__:__ACCOUNT_ID__:client/__THING_NAME__"
    },
    {
      "Sid": "PublishTelemetry",
      "Effect": "Allow",
      "Action": "iot:Publish",
      "Resource":
      "arn:aws:iot:__REGION__:__ACCOUNT_ID__:topic/psi5120-final/__ID__/lamp01/telemetry"
    },
    {
      "Sid": "PublishShadowUpdate",
      "Effect": "Allow",
      "Action": "iot:Publish",
      "Resource":
      "arn:aws:iot:__REGION__:__ACCOUNT_ID__:topic/$aws/things/__THING_NAME__/shadow/update"
    },
    {
      "Sid": "SubscribeShadowAndCommand",
      "Effect": "Allow",
      "Action": "iot:Subscribe",
      "Resource": [
        "arn:aws:iot:__REGION__:__ACCOUNT_ID__:topicfilter/$aws/things/__THING_NAME__/shadow/update/delta",
        "arn:aws:iot:__REGION__:__ACCOUNT_ID__:topicfilter/$aws/things/__THING_NAME__/shadow/update/accepted",
        "arn:aws:iot:__REGION__:__ACCOUNT_ID__:topicfilter/$aws/things/__THING_NAME__/shadow/update/rejected",
        "arn:aws:iot:__REGION__:__ACCOUNT_ID__:topicfilter/psi5120-final/__ID__/lamp01/command"
      ]
    },
    {
      "Sid": "ReceiveShadowAndCommand",
      "Effect": "Allow",
      "Action": "iot:Receive",
      "Resource": [
        "arn:aws:iot:__REGION__:__ACCOUNT_ID__:topic/$aws/things/__THING_NAME__/shadow/update/delta",
        "arn:aws:iot:__REGION__:__ACCOUNT_ID__:topic/$aws/things/__THING_NAME__/shadow/update/accepted",
        "arn:aws:iot:__REGION__:__ACCOUNT_ID__:topic/$aws/things/__THING_NAME__/shadow/update/rejected",
        "arn:aws:iot:__REGION__:__ACCOUNT_ID__:topic/psi5120-final/__ID__/lamp01/command"
      ]
    }
  ]
}

```

D. Telemetry Lambda Execution Role (inline policy)

Grants the telemetry-persistence function permission to write to DynamoDB only.

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PutTelemetryInDynamoDB",
      "Effect": "Allow",
      "Action": "dynamodb:PutItem",
      "Resource": "COLE_AQUI_O_ARN_DA_TABELA"
    }
  ]
}

```

E. Web Control Panel Lambda Execution Role (inline policy)

Grants the web control panel's backend function permission to read/update the Device Shadow and to publish the identify command only.

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ShadowReadWrite",
      "Effect": "Allow",
      "Action": [
        "iot:GetThingShadow",
        "iot:UpdateThingShadow"
      ],
      "Resource": "arn:aws:iot:__REGION__:__ACCOUNT_ID__:thing/__THING_NAME__"
    },
    {
      "Sid": "PublishIdentifyCommand",
      "Effect": "Allow",
      "Action": "iot:Publish",
      "Resource":
      "arn:aws:iot:__REGION__:__ACCOUNT_ID__:topic/psi5120-final/__ID__/lamp01/command"
    }
  ]
}

```